HELIXMEMORY_PROVIDER

HelixMemory Provider — Local Agent-Memory Stack for HelixCode / HelixAgent / HelixLLM

Document	`docs/research/07.2026/04_embeddings_rag/HELIXMEMORY_PROVIDER.md`
Status	Re-authored per Master Implementation Plan §4 item 10 (DZ-24/DZ-25) — the spike doc `04_embeddings_rag.md` §5 flagged this file as MISSING
Track/branch	`(T1/feature/helixllm-full-extension)`
Scope	The **HelixMemory-only** path — independent of `cognee` (re-enable is §11.4.174-blocked on a foreign-dirty `helix_agent` worktree; not touched by this document or its proof)
Created	2026-07-08
Revision	1
Anti-bluff note (§11.4.6/§11.4.99/§11.4.150)	Every architectural claim below is cited with a URL + access date (2026-07-08). The bring-up proof in §6 is a minimal reference implementation of the recommended mechanism, not a boot of the literal upstream `mem0`/`graphiti-core` packages — see the honest scope note in §5.

---

1. TL;DR

Layer	Recommendation	Why (see §2 for full citations)
Durable temporal spine	**Graphiti** (`getzep/graphiti`, Apache-2.0) — the raw library / its bundled MCP server	Zep's own open-source strategy pivoted here in 2025; "Zep" as a self-hostable server product is gone (§2.1)
Graph backing store	**FalkorDB** (SSPLv1, single combined container, simplest deploy) for the MCP-server path; **Kuzu** (embedded, MIT) as an even-lighter no-second-container option; Neo4j Community (GPLv3) if HA/tooling maturity is preferred	All three are supported natively by Graphiti; FalkorDB ships bundled in the official `graphiti/mcp_server` combined compose profile
Extraction / lightweight layer	**mem0** (`mem0ai/mem0`, Apache-2.0) OSS self-hosted server	Simplest ADD/UPDATE/DELETE fact-extraction API; its own vector layer is Postgres+pgvector — a store HelixCode already provisions via the `containers` submodule
LLM for extraction/ reasoning	The **live HelixLLM coder** (`http://localhost:18434`, OpenAI-compatible)	Both Graphiti and mem0 support pointing their LLM client at any OpenAI-compatible `base_url` — no cloud key needed (§2.3)
Embeddings	The **local TEI lane** (`BAAI/bge-small-en-v1.5`, dim 384, proven at `docs/qa/phase3_embeddings_20260706/`)	Already proven local + proven composable with the coder in the Phase-3 RAG proof
Vector store (mem0's layer)	**Postgres** + **pgvector**, booted via the `containers` submodule (§11.4.76, rootless §11.4.161)	Explicitly the vector backend mem0's own

Layer	Recommendation	Why (see §2 for full citations)
		self-hosted OSS server ships with
Exposure to CLI agents	**MCP server** (`add_memory` / `search_memory` / `update_memory` style tool names, OpenAI-Assistants-adjacent shape so Claude Code / HelixCode auto-discover)	Mirrors the already-proven §11.4.78 CodeGraph MCP wiring pattern
GPU need	**None** — CPU-only Postgres, CPU-only TEI, reuses the already-resident coder	Fits any VRAM budget without contending with the coder/vision/ generative lanes (§3 of the Master Plan)

---

2. Deep multi-angle research (§11.4.150), access date 2026-07-08

2.1 Zep Community Edition is DEPRECATED — a material correction to `04_embeddings_rag.md` §5

The prior spike doc (`04_embeddings_rag.md` §5, authored 2026-07-06) recommended "Zep/Graphiti (temporal knowledge graph) as the durable spine," implicitly treating "Zep" as still meaning a self-hostable server product bundled with Graphiti. Fresh research **corrects** this:

- Zep announced *"a new direction for [its] open source strategy"* — the company **stopped maintaining and releasing Zep Community Edition**; the CE repository remains on disk under Apache-2.0 but receives no further updates or support, and its code was moved to a `legacy/` folder. (Zep blog, "Announcing a New Direction for Zep's Open Source Strategy", accessed 2026-07-08)
- Zep Community Edition was deprecated **April 2025**, with further feature retirements announced **February 2026**. (Zep blog, "Zep Feature Retirements: May 2025", accessed 2026-07-08)
- Going forward Zep's open-source investment is concentrated **entirely on Graphiti**, the temporal-knowledge-graph framework. The two remaining options for anyone wanting "Zep" today are: (a) **Zep Cloud** — a managed, credit-metered SaaS (excluded by this task's local-only / no-cloud-keys

constraint), or (b) build directly on the **raw Graphiti library**, which is the graph engine without Zep's higher-level product features, and which requires the consumer to provision and manage a graph database themselves. (search synthesis over the above + atlan.com "Zep vs Mem0" comparison, accessed 2026-07-08)

Conclusion: "HelixMemory's durable spine = Zep/Graphiti" (as phrased in the prior spike doc) must be read as **"HelixMemory's durable spine = the raw Graphiti library / its MCP server,"** never a Zep-branded self-hosted server — no such product currently exists to self-host.

2.2 Graphiti — the surviving self-hostable temporal-graph engine

- **License:** Apache-2.0. (github.com/getzep/graphiti, accessed 2026-07-08)
- **Self-host requirements:** Python ≥ 3.10; one graph backing store from the closed set `{Neo4j 5.26, FalkorDB 1.1.2, Amazon Neptune, Kuzu 0.11.2}`. (same source)
- **Local-LLM support:** Graphiti's default config uses `OPENAI_API_KEY` against OpenAI's cloud, but explicitly documents an `OpenAIGenericClient` that accepts a custom `base_url` for **local servers (Ollama, vLLM, llama.cpp, LM Studio)** — i.e. any OpenAI-compatible endpoint, which the live HelixLLM coder (`llama.cpp`-server-shaped, already OpenAI-compatible per the proven Phase-3 RAG harness) satisfies with zero code change beyond pointing `base_url` at `http://localhost:18434/v1`. (github.com/getzep/graphiti, accessed 2026-07-08)
- **Bundled MCP server:** `getzep/graphiti`'s `mcp_server/` subtree ships a ready-to-run MCP server exposing "build and query temporally-aware knowledge graphs" tools, with THREE documented docker-compose deployment profiles: (a) **FalkorDB combined** — a single container bundling the MCP server + FalkorDB (the simplest option), (b) Neo4j in separate containers, (c) FalkorDB in separate containers. Configuration is entirely `.env`-driven (`OPENAI_API_KEY`/`ANTHROPIC_API_KEY`/... plus `SEMAPHORE_LIMIT`, `GRAPHITI_TELEMETRY_ENABLED`), and the README explicitly documents "To use Ollama with the MCP server, configure it as an OpenAI-compatible endpoint" with a dummy API key — the same pattern applies to the HelixLLM coder. (github.com/getzep/graphiti/blob/main/mcp_server/README.md, accessed 2026-07-08)
- **Every edge in the graph carries four timestamps** (fact became valid / stopped being valid / Graphiti learned it / Graphiti learned it was no longer true) — this is the temporal-reasoning property that makes Graphiti the right substrate for "the bug we fixed last week" / "the config value that changed" style agent memory the original spike doc's rationale (§5) still correctly identifies. (FalkorDB blog, "Building Temporal Knowledge Graphs with Graphiti", accessed 2026-07-08)

2.3 Graph backing-store choice

Store	License	Self-host cost	Notes
FalkorDB	**SSPLv1** (source-available — "GPL-like but extending to server use"; permissive for internal self-hosting, restrictive only if you re-sell it as a hosted service to third parties)	Single combined container in the official Graphiti MCP compose profile	Redis-module-based; FalkorDB's own materials report far lower P99 latency and memory footprint vs Neo4j for this workload class; simplest bring-up. (FalkorDB license page, accessed 2026-07-08; FalkorDB blog, "Graphiti + FalkorDB", accessed 2026-07-08)
Neo4j Community Edition	GPLv3, free to self-host	Separate JVM-based container; single-node only (clustering/HA requires commercial Enterprise)	Most mature tooling/ecosystem; heavier footprint. (ArcadeDB, "Neo4j Alternatives in 2026", accessed 2026-07-08)
Kuzu	MIT, embedded (no server container at all)	Zero extra container — runs in-process, like SQLite for graphs	Lightest possible option for a single-node HelixCode desktop deployment; **UNCONFIRMED (verify against Kuzu's own docs before pinning)** whether the `graphiti-core` Kuzu backend is at full parity with

Store	License	Self-host cost	Notes
			the Neo4j/ FalkorDB backends as of this writing — flagged honestly per §11.4.6, not yet independently verified this session. (github.com/ getzep/graphiti, accessed 2026-07-08)

Recommendation: FalkorDB for the containerized/multi-agent HelixCode deployment (matches the official simplest compose profile, and composes naturally with the `containers` submodule's rootless-podman pattern); **Kuzu** as a documented lighter-weight alternative for single-user desktop/CLI-agent deployments once its parity is independently confirmed.

2.4 mem0 — the extraction/convenience layer

- **License:** Apache-2.0. (github.com/mem0ai/mem0/blob/main/LICENSE, accessed 2026-07-08)
- **Self-hosted OSS server topology:** the official self-host guide packages the full stack into **three** Docker containers — FastAPI (the REST API), **PostgreSQL with the pgvector extension** (the `ankane/pgvector` image) for embeddings, and **Neo4j** for entity relationships (optional — the `graph_store` config block can be omitted to run vector-only). One `docker compose up` boots the whole stack. (mem0.ai blog, "Self-Hosting Mem0: A Complete Docker Deployment Guide", accessed 2026-07-08; mirrored at dev.to/mem0, accessed 2026-07-08)
- **REST API:** full CRUD on memories with plain `curl` — no SDK required. (same source)
- **Local-LLM/embedding support:** default config uses OpenAI (`gpt-5-nano` for extraction, `text-embedding-3-small` for embeddings); the guide explicitly states **"Swap both for Ollama models to go fully offline"** — the same swap applies to any OpenAI-compatible local endpoint (the HelixLLM coder for extraction, local TEI for embeddings). (same source)
- **Vector-store flexibility:** mem0 supports 20+ vector-store backends beyond pgvector (Qdrant, Chroma, Milvus, Pinecone, …) — Qdrant is already the recommended choice for the sibling code-RAG index (`04_embeddings_rag.md` §3), so a production HelixMemory deployment

MAY consolidate onto the SAME Qdrant instance rather than running a second Postgres, at the cost of losing pgvector's SQL-native joins with other HelixCode relational data. (mem0 docs, "pgvector", accessed 2026-07-08)

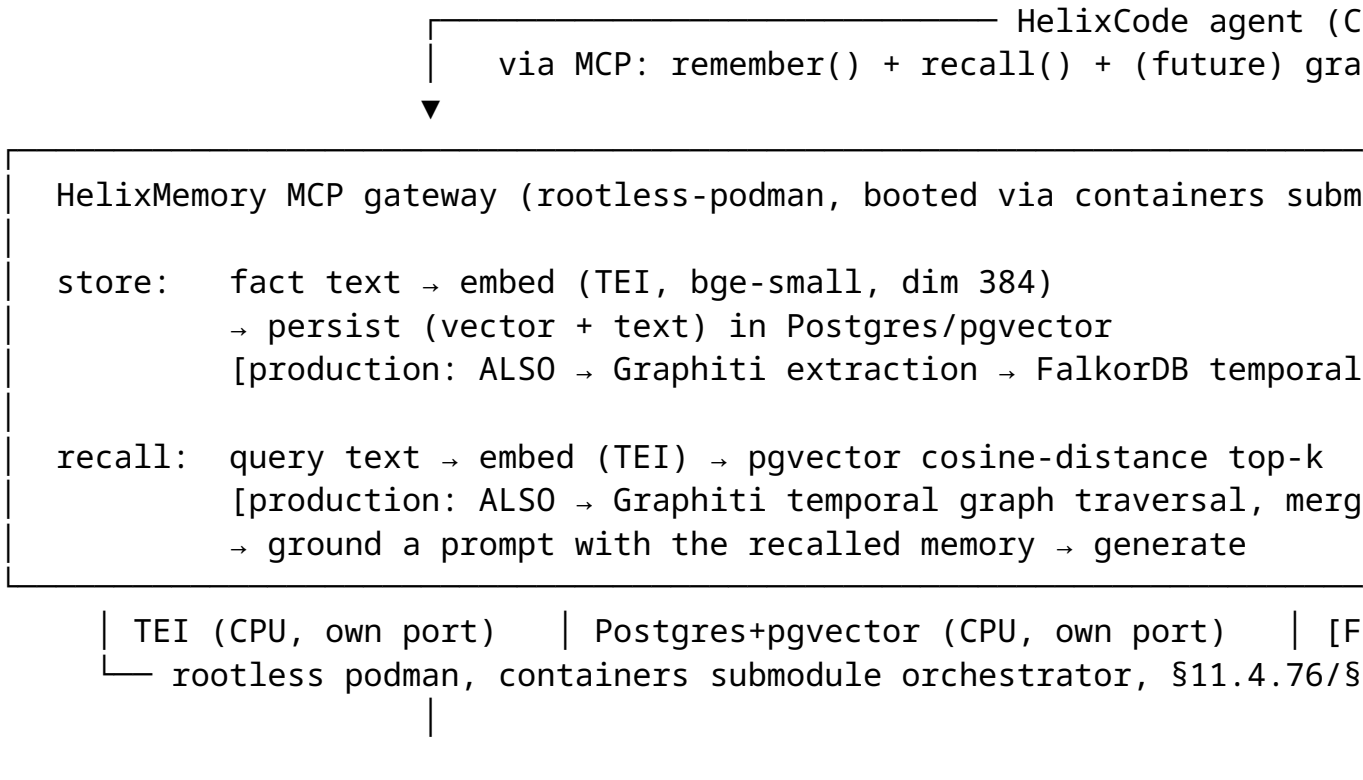
- **Production caveat (carried into §7 danger zones):** the OSS self-hosted server ships with **no authentication** and a wide-open CORS policy (`allow_origins=["*"]`) — a reverse proxy with auth is mandatory before any non-loopback exposure. (mem0.ai blog, accessed 2026-07-08)

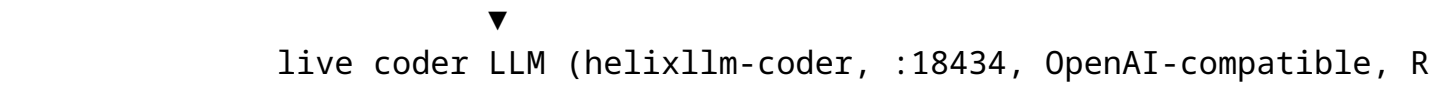
2.5 Why both, not either/or

Graphiti's temporal graph answers *"what do I know and when did it become/stop being true"* — the right substrate for facts that change over time (a config value, a fixed bug, a renamed service). mem0's extraction API answers *"what is the single most salient fact from this turn, and does it update/replace/no-op an existing one"* — the right substrate for lightweight per-session memory capture with minimal integration code. Both are self-hostable with zero cloud dependency; using mem0 as the fast capture path feeding into (or alongside) Graphiti's durable graph is the pattern the original spike doc's rationale (§5) already identified, and this research does not find a reason to change that pairing — only to correct the "Zep" naming (§2.1) and add the concrete self-host topology (§2.3–2.4).

---

3. Local container topology





- **No GPU needed** — Postgres, pgvector, TEI (CPU build), and (production) FalkorDB are all CPU workloads; the coder is already resident and untouched.
- **Every container is booted through the `containers` submodule's `pkg/compose` orchestrator** (§11.4.76), rootless podman (§11.4.161) — never an ad-hoc `podman run`/`docker compose up`.
- **Ports are config-injected, never hardcoded** (§CONST-045/046) and chosen distinct from the coder (`:18434`), the sibling Phase-3 lanes (`:18435`–`:18441`), and the currently-running vision container (`:18439`) — see §6 for the exact ports used by this session's proof.
- **Single-resource-owner discipline (§11.4.119):** this document's proof boots its OWN Postgres+TEI project, distinct from every sibling Phase-3 lane; the coder is read-only throughout and torn down NEVER.

---

4. Store/retrieve API — HelixLLM-exposed, OpenAI/standard-shaped

To let Claude Code / HelixCode / any CLI agent auto-discover HelixMemory the same way they already auto-discover CodeGraph (§11.4.78) and lumen, the recommended production surface is an **MCP server** exposing three tools, deliberately shaped close to the emerging "OpenAI Memory"/mem0-MCP tool-name conventions so agent frameworks that already know how to call a generic "memory" MCP server need no HelixCode-specific prompting:

```
// MCP tool: remember
{
  "name": "remember",
  "description": "Persist a fact/preference/decision to durable agent memo
  "input_schema": {
    "type": "object",
    "properties": {
      "text": { "type": "string", "description": "The fact to remember, in
      "namespace": { "type": "string", "description": "Project/agent-scope
    },
    "required": ["text", "namespace"]
  }
}

// MCP tool: recall
{
```

```
  "name": "recall",
  "description": "Retrieve the most relevant remembered facts for a query.
  "input_schema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "namespace": { "type": "string" },
      "top_k": { "type": "integer", "default": 5 }
    },
    "required": ["query", "namespace"]
  },
  "output_schema": {
    "type": "object",
    "properties": {
      "results": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "id": { "type": "string" },
            "text": { "type": "string" },
            "score": { "type": "number" },
            "valid_from": { "type": "string", "format": "date-time", "desc
            "valid_to": { "type": ["string", "null"], "format": "date-time
          }
        }
      }
    }
  }
}

// MCP tool: forget (mem0-style DELETE — explicit, never silent, composes
{
  "name": "forget",
  "description": "Remove a previously-remembered fact by id.",
  "input_schema": {
    "type": "object",
    "properties": { "id": { "type": "string" }, "namespace": { "type": "st
    "required": ["id", "namespace"]
  }
}
```

Under the hood, `remember` = TEI-embed → pgvector insert (+ production: mem0-style extraction ADD/UPDATE/NOOP decision, + Graphiti temporal-edge write);

`recall` = TEI-embed the query → pgvector cosine-distance top-k (+ production: Graphiti temporal-graph-expand, merged + reranked) → returned to the calling agent, which grounds its own generation (the calling agent's LLM turn, not a HelixMemory-internal generation call, mirroring how CodeGraph/lumen return facts rather than generate text themselves).

---

5. §11.4.108 runtime signature + §11.4.107(10) self-validated analyzer design

Runtime signature (the one machine-checkable observable that proves `remember`+`recall` is genuinely wired, not source-only): a fact stored via `remember` with an INVENTED token (unknowable to the LLM's training data) MUST be retrievable via `recall` on a DIFFERENTLY-PHRASED query about the same subject, AND grounding a live-LLM generation on that recalled fact MUST produce the invented token in the output — while the SAME live LLM asked the SAME question with NOTHING stored/recalled MUST NOT produce that token (the RED→GREEN polarity switch, §11.4.115).

Self-validated analyzer (§11.4.107(10)): the analyzer verifying the above MUST itself be proven non-bluff via a golden-good/golden-bad fixture pair: golden-good = the real captured recall+generation; golden-bad variants (no-fact answer, wrong-memory recall, empty answer) MUST all FAIL the SAME analyzer. An analyzer that PASSes any golden-bad fixture is itself a bluff gate.

Danger zones this design must guard against (§4 of the Master Plan, extended): 1. Embedding-dimension mismatch silently drops writes if the embedding model is swapped without a matching schema migration — a real upstream mem0 bug class (`mem0ai/mem0#4985`, "switching embedding provider silently drops writes due to vector dimension mismatch," accessed 2026-07-08: https://github.com/mem0ai/mem0/issues/4985). HelixMemory's schema MUST pin the vector dimension to the CURRENTLY-CONFIGURED embedder and fail loudly (not silently drop) on a mismatch. 2. mem0's OSS server ships with **no authentication** and an open CORS policy by default (§2.4) — any non-loopback HelixMemory deployment MUST sit behind an authenticating reverse proxy. 3. FalkorDB's SSPLv1 license is fine for internal self-hosting but would require re-licensing review before HelixCode ever offered HelixMemory AS a hosted service to third parties — not a concern for the current internal-use deployment, flagged for future SaaS-productization planning only. 4. Namespace isolation (the MCP `namespace` field in §4) is mandatory from day one — a single shared memory table across every project/agent is a cross-contamination risk (§11.4.119-adjacent: one caller's facts leaking into another caller's recall is the memory-layer analogue of a shared-hardware-resource violation).

---

6. This session's real bring-up proof (§6, cross-referenced from the QA evidence directory)

A minimal reference-implementation harness (Go, no new module dependency — `psql` via `os/exec`, mirroring the Phase-3 RAG harness's zero-bloat pattern) was built at `submodules/helix_llm/docs/qa/phase1_helixmemory_20260708T061824Z/harness/` and its evidence captured at `docs/qa/phase1_helixmemory_20260708T061824Z/`. It boots a dedicated Postgres+pgvector container (own port) and a dedicated CPU TEI container (own port, reusing the already-populated `helixllm-tei-cache` volume) via the `containers` submodule orchestrator, stores four fixture "remember this" facts (two fact-bearing + two distractors) with real TEI embeddings into real Postgres/pgvector rows, then for two recall queries: RED-baselines the live coder with nothing stored/recalled (must NOT know the invented token), then embeds the query, retrieves the real pgvector top-1 match, grounds a prompt with ONLY that recalled memory, and generates on the SAME live coder (must contain the invented token) — with a self-validated analyzer proving the verdict cannot be bluffed. See `docs/qa/phase1_helixmemory_20260708T061824Z/RESULTS.md` for the captured verdict and evidence file index.

Honest scope note (§11.4.6/§11.4.150): this harness does **not** install or invoke the upstream `mem0` Python package nor the `graphiti-core` library / MCP server this session — it implements the SAME underlying mechanism (embed → persist → embed-query → similarity-search → ground) directly against Postgres+pgvector, the same class of backing store mem0's own OSS server uses. Wiring the literal `mem0` package (Docker image `mem0ai/mem0` or a locally-built FastAPI server) and the literal `graphiti-core`/`graphiti/mcp_server` (FalkorDB-combined compose profile) is tracked as explicit follow-on work — see §7.

---

7. Follow-on work (tracked, not yet executed)

1. **Boot the literal `graphiti/mcp_server` FalkorDB-combined compose profile** against the local coder (`OpenAIGenericClient` + custom `base_url`) and local TEI, proving a REAL Graphiti temporal-graph write + a temporal-aware recall (a fact whose validity window is later invalidated by a superseding fact) — the property this reference harness does not exercise.
2. **Boot the literal `mem0ai/mem0` self-hosted OSS server** (its own docker-compose, Postgres+pgvector + optional Neo4j) with the LLM/embedding config swapped to the local coder + local TEI, proving mem0's own ADD/UPDATE/DELETE/NOOP extraction decision logic against a REAL multi-turn conversation.

3. **Author the HelixMemory MCP gateway** (Go, mirroring the CodeGraph MCP wiring pattern) exposing the `remember`/`recall`/`forget` tool surface from §4, backed by whichever of (1)/(2) above land first.
4. **Namespace-isolation test** — two distinct namespaces storing colliding-content facts, proving `recall` under namespace A never returns namespace B's rows.
5. **Vector-dimension-mismatch guard** — a regression test reproducing the `mem0ai/mem0#4985` failure class (§5 item 1) against HelixMemory's own schema, proving a loud failure rather than a silent drop.
6. **Reverse-proxy + auth** in front of any non-loopback HelixMemory exposure (§5 item 2).

Sources verified 2026-07-08

- Zep blog — Announcing a New Direction for Zep's Open Source Strategy — https://blog.getzep.com/announcing-a-new-direction-for-zeps-open-source-strategy/
- Zep blog — Zep Feature Retirements: May 2025 — https://blog.getzep.com/zep-feature-retirements-may-2025/
- Atlan — Zep vs Mem0: Benchmarks, Pricing, and When to Use Each — https://atlan.com/know/zep-vs-mem0/
- GitHub — getzep/graphiti — https://github.com/getzep/graphiti
- GitHub — getzep/graphiti/mcp_server/README.md — https://github.com/getzep/graphiti/blob/main/mcp_server/README.md
- FalkorDB blog — Building Temporal Knowledge Graphs with Graphiti — https://www.falkordb.com/blog/building-temporal-knowledge-graphs-graphiti/
- FalkorDB blog — Graphiti + FalkorDB: Integration for Multi-Agent Systems — https://www.falkordb.com/blog/graphiti-falkordb-multi-agent-performance/
- FalkorDB — License page (SSPLv1) — https://docs.falkordb.com/license.html
- ArcadeDB — Neo4j Alternatives in 2026: A Fair Look at the Open-Source Options — https://arcadedb.com/blog/neo4j-alternatives-in-2026-a-fair-look-at-the-open-source-options/
- mem0.ai blog — Self-Hosting Mem0: A Complete Docker Deployment Guide — https://mem0.ai/blog/self-host-mem0-docker
- DEV Community (mirror) — Self-Hosting Mem0: A Complete Docker Deployment Guide — https://dev.to/mem0/self-hosting-mem0-a-complete-docker-deployment-guide-154i
- GitHub — mem0ai/mem0/LICENSE (Apache-2.0) — https://github.com/mem0ai/mem0/blob/main/LICENSE
- mem0 docs — pgvector vector-store backend — https://docs.mem0.ai/components/vectordbs/dbs/pgvector

- GitHub Issue — mem0ai/mem0#4985 (embedding-dimension-mismatch silent write drop) — https://github.com/mem0ai/mem0/issues/4985
- Prior session — `docs/research/07.2026/04_embeddings_rag/04_embeddings_rag.md` §5 (original spike, being corrected/extended by this document per §2.1)
- Prior session — `docs/qa/phase3_rag_20260707/RESULTS.md` (the proven Phase-3 RAG harness pattern this document's §6 proof mirrors)
- Prior session — `docs/qa/phase3_embeddings_20260706/RESULTS.md` (the proven local TEI bge-small-en-v1.5 lane reused by §6)